Conceptos y fundamentos de JDBC

1.1 Introducción a JDBC

JDBC (Java Database Connectivity) es una API estándar de Java que permite conectar una aplicación con una base de datos relacional (como MySQL, PostgreSQL o SQLite).

Su **objetivo** es abstraer la comunicación: el desarrollador escribe código Java que usa interfaces JDBC, y el driver JDBC convierte esas llamadas en comandos del sistema de base de datos. Es decir, su función es **traducir las órdenes SQL** que escribimos en Java para que la base de datos las entienda y las ejecute correctamente.

Permite portar la lógica de acceso a datos sin depender del sistema específico de base de datos (siempre que haya driver JDBC disponible).

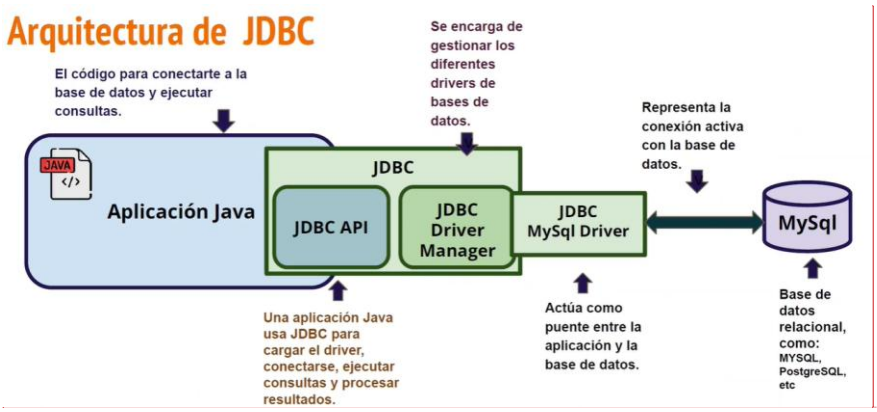
Sin JDBC, cada base de datos requeriría su propio lenguaje o protocolo. JDBC actúa como **intermediario** entre la aplicación Java y el sistema gestor de base de datos (SGBD).

Arquitectura JDBC

La arquitectura de JDBC sigue un modelo de cuatro capas principales:

1. **Aplicación Java:** el programa que realiza operaciones sobre la base de datos.
2. **API JDBC:** clases e interfaces que proporciona Java (java.sql y javax.sql).
3. **Driver JDBC:** componente que traduce las peticiones JDBC al lenguaje del gestor de base de datos.
4. **SGBD:** la base de datos propiamente dicha (MySQL, PostgreSQL, SQLite, etc.).

Capa	Descripción	Ejemplo
Aplicación Java	El programa que realiza las operaciones.	Clase Main o App
API JDBC	Conjunto de interfaces del paquete java.sql.	Connection, Statement, ResultSet
Driver JDBC	Traduce las órdenes JDBC a un lenguaje que la base de datos entienda.	com.mysql.cj.jdbc.Driver
SGBD	Sistema Gestor de Base de Datos.	MySQL, PostgreSQL, SQLite



Comentado [L1]: https://www.youtube.com/watch?v=StqvFxkoxwU
En este vídeo puedo completar contenido

1.2. Tipos de drivers

Los **drivers JDBC** son los componentes que permiten a una aplicación Java comunicarse con un sistema gestor de bases de datos (SGBD). Cada driver actúa como un "traductor" que convierte las peticiones realizadas a través de la API JDBC en instrucciones que el SGBD puede entender y ejecutar.

En función de cómo se implemente esa traducción, existen cuatro tipos de drivers. Antes de verlos en detalle, conviene comprender los **tipos de bases de datos** con los que una aplicación Java puede trabajar, ya que esta clasificación influye directamente en el driver que utilizaremos.

Tipos de bases de datos en función de su conexión

Gestores embebidos:

- Se ejecutan dentro de la propia aplicación, sin necesidad de un servidor externo.
- Son ideales para aplicaciones pequeñas o portátiles, ya que almacenan los datos en un único archivo (.db) y no requieren instalación adicional
- El driver JDBC se carga directamente desde el propio programa y se conecta al archivo de base de datos.
- Ejemplo: **SQLite**.

Gestores independientes:

- Funcionan como un servicio o servidor al que la aplicación se conecta mediante red (localhost o IP).
- Permiten múltiples usuarios concurrentes, conexiones simultáneas y escalabilidad
- Se utilizan en entornos cliente–servidor y aplicaciones web.
- Ejemplos: **MySQL**, **PostgreSQL**.

Existen **cuatro tipos de drivers**, aunque en la práctica actual se utilizan principalmente los de tipo 4.

Tanto para los SGBD embebidos como independientes la conexión desde Java se realiza mediante **drivers JDBC tipo 4 (más adelante se explican)**, aunque la **URL y el driver concreto** cambian según el gestor utilizado.

Veamos los 4 drivers posibles:

Tipo	Descripción	Ventajas	Inconvenientes
Tipo 1 (JDBC-ODBC Bridge)	Usa un puente ODBC intermedio	Simple	Obsoleto, requiere ODBC
Tipo 2 (Native API)	Usa una API nativa del SGBD	Rápido	Depende del sistema operativo
Tipo 3 (Network Protocol)	Usa un middleware intermedio	Independiente del cliente	Necesita servidor intermedio
Tipo 4 (Pure Java Driver)	Implementado 100% en Java	Multiplataforma, rápido, recomendado	Ninguno relevante

Tipo 1: JDBC–ODBC Bridge (obsoleto)

Descripción: usaba un *puente ODBC* intermedio. El driver JDBC traducía las llamadas a ODBC, y este se comunicaba con el gestor de base de datos mediante un *driver ODBC* instalado en el sistema.
Estado actual: Eliminado a partir de Java 8. Solo se usa con versiones antiguas (JDK 6 o 7).

```
import java.sql.*;

public class DriverTipo1 {
    public static void main(String[] args) {
        try {
            // Cargar el puente JDBC-ODBC (obsoleto desde Java 8)
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

            // URL ODBC configurada en el sistema (DSN)
            String url = "jdbc:odbc:miDSNMySQL";
            Connection con = DriverManager.getConnection(url, "root",
"Password1234");
            System.out.println("Conexión establecida con Driver Tipo 1");
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Tipo 2: Driver Nativo (Native API)

Descripción: utiliza librerías nativas específicas del sistema operativo o del SGBD. Parte del código es Java, pero requiere dependencias nativas.

Estado actual: Muy raro hoy en día; dependía del sistema operativo y complicaba la portabilidad.

```
import java.sql.*;

public class DriverTipo2 {
    public static void main(String[] args) {
        try {
            // Cargar driver nativo (ejemplo teórico)
            Class.forName("com.mysql.jdbc.DriverNative");
            Connection con = DriverManager.getConnection(
                    "jdbc:mysql:native://localhost:3306/empresa",
                    "root", "Password1234");
            System.out.println("Conexión establecida con Driver Tipo 2");
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Tipo 3: Driver de Protocolo de Red (Middleware)

Descripción: el driver JDBC no se conecta directamente al SGBD, sino a un **servidor intermedio** (middleware) que traduce las peticiones hacia MySQL. Era útil en entornos corporativos antes de la aparición de los drivers tipo 4.

Estado actual: En desuso, sustituido por drivers tipo 4 y frameworks modernos.

```
import java.sql.*;

public class DriverTipo3 {
    public static void main(String[] args) {
        try {
            // Driver de red genérico (ejemplo teórico)
            Class.forName("com.middleware.jdbc.NetworkDriver");

            // El middleware escucha en el puerto 7000 y reenvía al SGBD
MySQL
            String url = "jdbc:middleware://localhost:7000/empresa";
            Connection con = DriverManager.getConnection(url, "root",
"Password1234");
            System.out.println("Conexión establecida con Driver Tipo 3
(middleware)");
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Tipo 4: Driver Puro Java (actual y recomendado)

Descripción: implementado completamente en Java. Traduce las llamadas JDBC directamente a comandos nativos del gestor de base de datos.

Estado actual: Es el que usamos hoy en día con MySQL, PostgreSQL, SQLite, etc.

Ventajas del tipo 4:

- 100% Java → multiplataforma
- No requiere componentes externos
- Rápido y eficiente
- Es el estándar actual

Ejemplos:

- MySQL → com.mysql.cj.jdbc.Driver
- PostgreSQL → org.postgresql.Driver
- SQLite → org.sqlite.JDBC

A continuación, se muestra un ejemplo de este tipo para estas tres bases de datos.

Ejemplo 1: Conexión JDBC Tipo 4 con MySQL

Driver: com.mysql.cj.jdbc.Driver
Librería necesaria: mysql-connector-j-8.0.xx.jar *(disponible en Maven Central o en la web oficial de MySQL)*

Dependencia en pom.xml:

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.0</version> <!-- o la más reciente disponible -->
</dependency>
```

Fichero java:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class ConexionMySQL {
    public static void main(String[] args) {
        // Datos de conexión
        String url = "jdbc:mysql://localhost:3306/empresa";
        String user = "root";
        String password = "Password1234";

        // Conexión usando try-with-resources
        try (Connection con = DriverManager.getConnection(url, user,
password);
             Statement stmt = con.createStatement()) {
```

```
            System.out.println("Conexión establecida con MySQL (driver tipo
4).");

            // Crear una tabla de ejemplo
            String sql = "CREATE TABLE IF NOT EXISTS empleados (" +
                         "id INT AUTO_INCREMENT PRIMARY KEY, " +
                         "nombre VARCHAR(50), " +
                         "salario DECIMAL(10,2))";
            stmt.execute(sql);
            System.out.println("Tabla creada correctamente.");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo 2: Conexión JDBC Tipo 4 con SQLite

Driver: org.sqlite.JDBC

Librería necesaria: sqlite-jdbc-x.x.x.jar

(se puede descargar de https://github.com/xerial/sqlite-jdbc)

Observación: SQLite no necesita usuario ni contraseña.
Todo se guarda en un único fichero .db, por eso es ideal para aplicaciones pequeñas o portátiles.

Dependencia fichero pom.xml:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.46.1.0</version> <!-- versión actualizada 2025 -->
</dependency>
```

Fichero Java:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class ConexionSQLite {
    public static void main(String[] args) {
        // Ruta de la base de datos (se crea si no existe)
        String url = "jdbc:sqlite:C:/basedatos/empresa.db";

        try (Connection con = DriverManager.getConnection(url);
             Statement stmt = con.createStatement()) {

            System.out.println("Conexión establecida con SQLite (driver tipo
4).");

            // Crear una tabla de ejemplo
            String sql = "CREATE TABLE IF NOT EXISTS empleados (" +
                         "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                         "nombre TEXT, " +
                         "salario REAL)";
```

```
            stmt.execute(sql);
            System.out.println("Tabla creada correctamente.");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo 3: Conexión JDBC Tipo 4 con PostgreSQL

Driver: org.postgresql.Driver

Librería necesaria: postgresql-x.x.x.jar

(se puede obtener desde Maven Central o el sitio oficial de PostgreSQL)

Dependencia fichero pom.xml:

```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.3</version> <!-- versión estable 2025 -->
</dependency>
```

Fichero Java:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionPostgreSQL {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://localhost:5432/empresa";
        String user = "postgres";
        String password = "Password1234";

        try (Connection con = DriverManager.getConnection(url, user,
password)) {
            System.out.println("Conexión establecida con PostgreSQL (driver
tipo 4).");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Comparativa de los tres ejemplos anteriores:

Base de datos	Driver principal (Java)	URL de conexión	Dependencia Maven	Servidor requerido	Observaciones
MySQL	com.mysql.cj.jdbc.Driver	jdbc:mysql://localhost:3306/empresa	com.mysql:mysql-connector-j:9.0.0	Sí	Muy usado en aplicaciones web y backend.
PostgreSQL	org.postgresql.Driver	jdbc:postgresql://localhost:5432/empresa	org.postgresql:postgresql:42.7.3	Sí	Estándar libre, excelente para proyectos empresariales.
SQLite	org.sqlite.JDBC	jdbc:sqlite:C:/basedatos/empresa.db	org.xerial:sqlite-jdbc:3.46.1.0	No	Base de datos embebida; crea un archivo .db local.

Nota: en proyectos Maven no es necesario añadir el .jar manualmente, ya que el conector se descarga automáticamente desde el repositorio central al compilar el proyecto

Para finalizar este apartado, concluimos que los cuatro tipos de drivers JDBC se **refieren al modo técnico** en que Java se comunica con la base de datos. Sobre esta base **se construyen tecnologías de persistencia más avanzadas** (como Hibernate, MyBatis o Spring Data JPA), que simplifican el trabajo del programador, pero internamente siguen usando un driver JDBC (normalmente de tipo 4) para establecer la conexión real con el SGBD.

Tipo	Nombre	Traducción de llamadas	Requiere componentes externos	Estado actual
1	JDBC–ODBC Bridge	JDBC → ODBC → SGBD	Driver ODBC del sistema	Obsoleto
2	Native API	JDBC → librería nativa → SGBD	Librerías nativas	Obsoleto
3	Network Protocol	JDBC → middleware → SGBD	Servidor intermedio	En desuso
4	Pure Java	JDBC → SGBD (directo)	Ninguno	En uso (recomendado)

1.3 Tecnologías comunes para la persistencia en Java

A partir de la capa básica de **JDBC**, existen distintas tecnologías y frameworks que simplifican la gestión de datos y reducen el código necesario para acceder a las bases de datos.

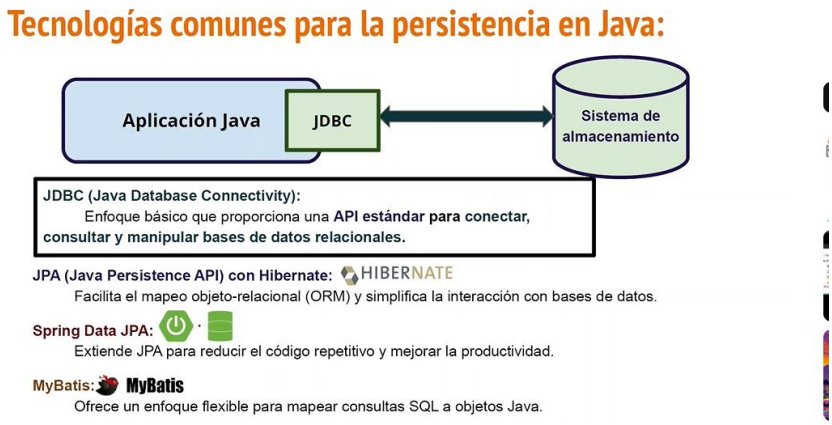
Estas herramientas se construyen **encima de JDBC**, por lo que internamente siguen utilizando un **driver tipo 4** para comunicarse con el sistema gestor de base de datos.

Tecnología	Descripción	Nivel de abstracción
JDBC	API estándar de Java para conectar, consultar y manipular bases de datos relacionales.	Bajo
JPA / Hibernate	Simplifica la interacción con la base de datos mediante mapeo objeto-relacional (ORM).	Medio
Spring Data JPA	Extiende JPA y automatiza operaciones CRUD, reduciendo código repetitivo.	Alto
MyBatis	Permite mapear consultas SQL personalizadas a objetos Java de forma flexible.	Medio

Siguiente unidad de trabajo

No se incluye en el currículo MUY IMPORTANTE

Nota: Estas tecnologías no sustituyen a JDBC, sino que lo utilizan internamente. El programador elige el nivel de abstracción adecuado según la complejidad del proyecto.



1.4 Flujo típico de una operación JDBC

El trabajo con JDBC sigue siempre la misma secuencia de pasos, independientemente del sistema gestor de bases de datos (MySQL, SQLite o PostgreSQL).
A continuación se explica el flujo general acompañado de ejemplos para cada SGBD.

Paso 1. Registrar o inicializar el driver JDBC

Permite que Java reconozca **el tipo de base de datos** con el que se va a comunicar. Hoy en día este paso suele ser **implícito** (el driver se carga automáticamente al ejecutar el programa), pero puede hacerse manualmente usando Class.forName().

Ejemplo correspondiente para cada base de datos:

```
// MySQL
Class.forName("com.mysql.cj.jdbc.Driver");

// SQLite
Class.forName("org.sqlite.JDBC");

// PostgreSQL
Class.forName("org.postgresql.Driver");
```

En la práctica moderna (drivers tipo 4), esta línea no suele ser necesaria si el driver está correctamente incluido en el proyecto Maven.

Paso 2. Obtener una conexión

Se utiliza DriverManager.getConnection() indicando la **URL de conexión**, el **usuario** y la **contraseña** (excepto en SQLite, que no los requiere).
Este método devuelve un objeto Connection, que será el punto de partida para todas las operaciones con la base de datos.

```
// MySQL
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/empresa", "root", "Password1234");

// SQLite
Connection con = DriverManager.getConnection(
    "jdbc:sqlite:C:/basedatos/empresa.db");

// PostgreSQL
Connection con = DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/empresa", "postgres", "Password1234");
```

Nota: Si la conexión es correcta, no se lanzará excepción y se podrá crear una sentencia SQL.

Paso 3. Crear una sentencia SQL

Una vez establecida la conexión, se crea una sentencia con Statement o PreparedStatement. Con PreparedStatement se pueden usar parámetros para consultas más seguras y eficientes.

Ejemplo:

```
// MySQL
PreparedStatement pst = con.prepareStatement("INSERT INTO empleados (nombre,
salario) VALUES (?, ?)");
pst.setString(1, "Ana");
pst.setDouble(2, 1500.00);

// SQLite
PreparedStatement pst = con.prepareStatement("INSERT INTO empleados (nombre,
salario) VALUES (?, ?)");
pst.setString(1, "Luis");
pst.setDouble(2, 1200.50);

// PostgreSQL
PreparedStatement pst = con.prepareStatement("INSERT INTO empleados (nombre,
salario) VALUES (?, ?)");
pst.setString(1, "María");
pst.setDouble(2, 1800.75);
```

Paso 4. Ejecutar la sentencia

El método que se usa depende del tipo de operación SQL:

- executeUpdate() → para **INSERT**, **UPDATE**, **DELETE** o **CREATE TABLE**.
- executeQuery() → para **SELECT**.

Ejemplo:

```
// Ejecución para los tres casos
int filasAfectadas = pst.executeUpdate();
System.out.println("Filas insertadas: " + filasAfectadas);
```

Nota: Todos los SGBD responden del mismo modo; solo cambia la URL y el driver utilizados.

Paso 5. Procesar los resultados

Cuando la operación es una consulta (SELECT), se usa un ResultSet para recorrer los registros obtenidos.

Ejemplo:

```
// MySQL / SQLite / PostgreSQL (idéntico código)
PreparedStatement pst = con.prepareStatement("SELECT * FROM empleados");
ResultSet rs = pst.executeQuery();

while (rs.next()) {
    System.out.println("ID: " + rs.getInt("id") +
                       " | Nombre: " + rs.getString("nombre") +
                       " | Salario: " + rs.getDouble("salario"));
}
```

Paso 6. Cerrar los recursos

Cerrar siempre los objetos abiertos (ResultSet, PreparedStatement, Connection). La forma recomendada es usar **try-with-resources**, que cierra automáticamente al salir del bloque.

```
try (Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/empresa", "root", "Password1234");
     PreparedStatement pst = con.prepareStatement("SELECT * FROM empleados");
     ResultSet rs = pst.executeQuery()) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

Paso 7.Gestionar transacciones (opcional)

Si se desea realizar varias operaciones que deban ejecutarse todas o ninguna, se puede desactivar el modo automático de confirmación (autoCommit) y controlar manualmente commit() o rollback().

```
// MySQL / PostgreSQL
try (Connection con = DriverManager.getConnection(
        "jdbc:postgresql://localhost:5432/empresa", "postgres",
"Password1234")) {

    con.setAutoCommit(false); // Desactivar autocommit

    PreparedStatement pst1 = con.prepareStatement(
        "UPDATE empleados SET salario = salario + 100 WHERE nombre = 'Ana'");
    pst1.executeUpdate();

    PreparedStatement pst2 = con.prepareStatement(
        "INSERT INTO log_operaciones (accion) VALUES ('Subida salarial')");
    pst2.executeUpdate();

    con.commit(); // Confirmar cambios
    System.out.println("Transacción completada correctamente.");

} catch (SQLException e) {
    e.printStackTrace();
    // con.rollback(); // Deshacer si algo falla
}
```

SQLite también admite transacciones, aunque su control es más limitado en entornos concurrentes.

1.5 Buenas prácticas

Usa **try-with-resources** para cerrar automáticamente conexiones y liberar recursos.

No guardes contraseñas directamente en el código (usa ficheros de configuración).

Valida que el driver está incluido en el **classpath** del proyecto.